

Yaşar University
Spring, 2024 - 2025
SE 2232 - Software System Analysis
Final Project Report: Software Requirements Specifications Document (SRS)

Student Name:	Batıkan Akdeniz Zeynep Yıldız
Student No:	22070006053 22070006036
Department Name:	Software Engineering
Course Section No:	SE 2232 Section 2

Table of Contents (Do not change the Section Names!)

1	Introduction.....	3
1.1	Purpose	3
1.2	Scope	3
1.3	Definitions, acronyms, and abbreviations	3
1.4	References.....	4
1.5	Overview	4
2	Design and Implementation Constraints.....	5
3	Specific Requirements.....	5
3.1	Functional Requirements	5
3.2	Performance Requirements	6
3.3	Software System Attributes	7
3.4	Use Case Analysis.....	7
3.4.1	Actors	7
3.4.2	Scenarios	7
3.4.3	Use Case Forms.....	7
3.4.4	Relationships among Actors and Use Cases	13
3.4.5	Use Case Diagram	14
4	Behavioral Models.....	15
4.1	Sequence Diagram	15
5	Structural Models.....	18
5.1	Class Diagram	18
6	Process Modeling	19
6.1	Data Flow Diagram (DFD)	19
7	Graphical User Interface(s) (GUIs)	22
8	Conclusion and Future Work	30

1 Introduction

1.1 Purpose

The purpose of this project is to develop a personalized book management system that allows users to store, categorize, and manage their book collection based on reading status and individual preferences.

1.2 Scope

- a) The software product to be developed is named **MyLibrary Book System**.
- b) This software will allow users to manage their personal library by adding, editing, searching, and organizing books. Each book entry includes detailed information such as title, author, reading status (read, not read, wish to read), rating, comments, and release date. The system also handles author details and user-specific preferences like wish lists and favorites. Two types of users are supported: Type-1 User(Admin) users (with full access) and Standard users (with limited functionality).
- c) The goal of the MyLibrary application is to provide users with a simple yet powerful tool to track their reading habits, organize books effectively, and make informed reading decisions. For developers, it also demonstrates the integration of Java (Swing GUI) and MySQL using fundamental principles of software system analysis and design.

1.3 Definitions, acronyms, and abbreviations

Term / Acronym	Meaning
GUI	Graphical User Interface
SRS	Software Requirements Specification
IDE	Integrated Development Environment
DFD	Data Flow Diagram
UML	Unified Modeling Language
SHA-256	Secure Hash Algorithm 256-bit (cryptographic password hashing function)
Admin	Administrator (user with full privileges)
DB / Database	A structured collection of data
Book ID	Unique identifier for a book
Author ID	Unique identifier for an author
Type-1 User	User with administrative privileges

Term / Acronym	Meaning
Type-2 User	Regular user with limited access
SQL	Structured Query Language
CRUD	Create, Read, Update, Delete – basic database operations
Login	Process of user authentication
MyLibrary	Name of the software system/project
Alt / Opt Fragments	Elements used in UML sequence diagrams to represent conditional logic

1.4 References

1.5 Overview

This Software Requirements Specification (SRS) document clearly explains all parts of the MyLibrary Book Management System. It is organized in a way that helps guide the development process and makes sure everyone involved understands what the system is supposed to do and what its main goals are.

The report is organized into different sections to make it easy to follow:

- **Section 1** gives general information about the project. It explains the aim, what the system will cover, terms used in the report, and this overview.
- **Section 2** talks about the design rules and technical limitations, such as which programming languages and tools will be used (Java, MySQL, Swing, etc.).
- **Section 3** includes all the detailed requirements of the system. This part explains what features the system should have (functional requirements), how fast or reliable it should be (performance), and user scenarios with diagrams to show how the system will be used.
- **Section 4** shows the behavior of the system with a **sequence diagram**, which helps to visualize how users and the system interact step by step.
- **Section 5** presents the system's structure using a **class diagram**, which shows the classes (such as Book, Author, User) and their relationships.
- **Section 6** explains how the system processes data using a **Data Flow Diagram (DFD)**. This helps to understand how information moves inside the system.
- **Section 7** provides screenshots or sketches of the **Graphical User Interface (GUI)** to show how the application will look and work for users.
- **Section 8** gives a conclusion and suggestions for possible improvements that could be made in the future.

2 Design and Implementation Constraints

The development of the **MyLibrary Book System** is subject to several design and implementation constraints due to the technologies and tools selected for this project.

- **Development Environment:**
The system is being developed using **Apache NetBeans IDE 25**, which serves as the primary Integrated Development Environment (IDE) for building and testing the Java application.
- **Programming Language:**
The entire application is written in **Java**, which defines the structure and syntax of the software and limits the selection of certain libraries and platform-specific features.
- **Database System:**
MySQL Workbench version 8.0.42 (Build 5076204 CE, 64-bit) is used for designing, managing, and querying the relational database that stores all persistent data. The version constraints may limit compatibility with future updates or certain SQL functions.
- **Modeling Tool:**
All system modeling diagrams, including use case diagrams, class diagrams, sequence diagrams, and data flow diagrams, are created using **Visual Paradigm version 17.1**.
- **Platform Limitations:**
The project is designed and tested primarily on **Windows-based** and local **Apple MacBook (macOS)** machines. Cross-platform compatibility and deployment to different operating systems are not within the current scope.

These constraints must be considered during the design, development, and testing phases to ensure the system functions properly within the selected environment.

3 Specific Requirements

3.1 Functional Requirements

- The system shall allow users to log in by providing a valid username and password.
- The system shall notify the user if the release date of a book marked as 'wish to read' is within the next 7 days.
- The system shall determine the user type (Type-1 User(Admin) or Type-2 User) upon login and open the appropriate GUI frame.

- The system shall allow Type-1 User(Admin) users to add a new book by entering all required book attributes using text fields.
- The system shall prevent adding a book with a duplicate book ID.
- The system shall prevent users from assigning a rating to books that are marked as 'not read' or 'wish to read' (read = 2 or 3).
- The system shall prevent adding a book if the title field is left empty.
- The system shall prevent setting a release date that is in the past.
- The system shall automatically insert a new author if the entered author does not already exist in the system.
- The system shall generate the website field of a newly inserted author in the format "website-authorId" after the authorId is assigned.
- The system shall allow Type-1 User(Admin) users to display the detailed information of a book by entering its Book ID.
- The system shall allow Type-1 User(Admin) users to remove a book by its Book ID.
- The system shall delete the author entry if the removed book was the only book written by that author.
- The system shall allow Type-1 User(Admin) users to update a book by first retrieving it using its Book ID and then editing its attributes in a new update page.
- The system shall allow users (Type-1 User(Admin) and limited) to search for an author by entering the author's name and surname.
- The system shall display the author's information if the author exists in the database; otherwise, it shall display a message indicating that the author could not be found.
- The system shall display the list of books marked as "read" and rated 4 or 5 as favorite books.
- The system shall allow users to search for books using keywords from the title, author name, or description.
- The system shall display search results in a readable list or table format after a successful search query.

3.2 Performance Requirements

- a) The system shall handle a database containing up to 5,000 book records and 1,000 author without performance degradation.
- b) The system shall retrieve and display search results for a book title within 3 seconds, even with a database containing 10,000 records.
- c) The system shall respond to user actions (such as search author by name or login) within 2 seconds under network conditions.

3.3 Software System Attributes

Usability: The system shall have an easy-to-use interface with clear buttons and text fields so that users can use it without needing any training.

Security: The system shall store all user passwords in the database using secure hashing algorithms which is “SHA-256” to protect user credentials.

Maintainability: The system shall be developed with independent and loosely-coupled classes so that changes in one class do not affect the functionality of others.

Reliability: The system shall ensure that all user data, including book records and author information, is saved accurately without loss during normal operations and unexpected shutdowns.

Availability: The system shall be accessible to users 99% of the time during regular working hours.

3.4 Use Case Analysis

3.4.1 Actors

-Type-1 User(Admin)(Type-1 User)

-User(Type-2 User)

3.4.2 Scenarios

- 1) **Scenerio Name:** SearchAnAuthorByName

Scenario:

The user logs into the system. The system redirects the user to the appropriate interface based on their user type. The user clicks the button labeled "**Search an Author by Name.**" The user enters the author's name and surname into the appropriate fields. If the author exists, the system displays the author's information.

If not, the system displays the message: "**Author could not be found.**"

- 2) **Scenerio name:** UpdateBook

Scenario:

The user logs into the system with a correct username and password. The system redirects the user to the Type-1 User(Admin) page. The Type-1 User(Admin) clicks the "**Update Book**" button. A new screen opens, and the Type-1 User(Admin) enters the **Book ID** of the book they want to update. If the Book ID exists, the system displays the current book information. The Type-1 User(Admin) can then modify any field except the Book ID. If there is no matching book, the system displays the message "**The book is**

not found." After that, the Type-1 User(Admin) can return to the main menu or log out of the system.

3.4.3 Use Case Forms

■ Complete Use Case Form 1

Use case name: Add Book

Participating actor: Type-1 User(Admin)

Description: This use case allows the Type-1 User(Admin) to add a new book to the system by entering all required attributes such as title, author, year, number of pages, read status, and release date. If the author does not already exist in the database, the system inserts the author and auto-generates their website information.

Trigger: The Type-1 User(Admin) clicks the "ADD BOOK" button from the main interface.

Preconditions:

- The User must be successfully logged into the system.
- All input fields must be filled with correct data form.
- User Type must be Type-1(Type-1 User(Admin)).

Normal Course:

- Type-1 User(Admin) clicks the "ADD BOOK" button.
- A form opens for book information entry.
- Type-1 User(Admin) enters the book's title, author name and surname, year, number of pages, cover, about, read, status, rating (if applicable), comments, and release date.
- The system checks if the Book ID already exists.
- If the Book ID is unique, the system checks if the author exists.

- If the author does not exist, the system inserts the author with a temporary website and updates it to "website-authorId".

- The system inserts the book record into the database.

- A confirmation message is displayed.

Postcondition:

- The book is added to the database.

- The author is added if they were not already in the database.

Exceptions:

- If the title field is empty, the system prevents submission and shows a warning.

- If the release date is in the past, the system rejects the entry.

- If the Book ID already exists, the system cancels the operation and shows an error message.

- If any database error occurs during insert, the system displays a failure message.

- Rating cannot be entered if the book is not marked as read.

■ **Complete Use Case Form 2**

Use case name: Delete Book

Participating actors: Type-1 User(Admin)x

Description: This use case allows the Type-1 User(Admin) to delete a book from the system by entering the Book ID. If the book is the only one written by its author, the system also deletes the author from the database.

Trigger: The Type-1 User(Admin) clicks the " REMOVE BOOK " button from the main interface.

Preconditions:

- The Type-1 User(Admin) must be logged in.

- A valid Book ID must be entered.

Normal Course:

- Type-1 User(Admin) clicks the “REMOVE BOOK” button.
- A prompt appears for entering the Book ID.
- Type-1 User(Admin) enters the Book ID and confirms the action.
- The system checks whether the Book ID exists.
- If the book exists, the system deletes the book from the database.
- The system checks if the related author has any remaining books.
- If the author has no other books, the system deletes the author.
- The system displays a success message indicating the book (and possibly the author) was deleted.

Postcondition:

- The selected book is removed from the database.
- The related author is also removed if no other books remain by that author.

Exceptions:

- If the entered Book ID does not exist, the system displays an error message: "**Book not found.**"

■ Complete Use Case Form 3

Use case name: Display Book Cover Image

Participating actors: Type-1 User(Admin), Type-2 User

Description: This use case allows a user to view the cover image of a specific book by entering its Book ID. The system retrieves and displays the image stored in the database.

Trigger: The user clicks the “DISPLAY BOOK’S COVER” button and enters a valid Book ID.

Preconditions:

- The user must be logged into the system.

- A valid Book ID must be entered.

Normal Course:

- The user clicks the " DISPLAY BOOK'S COVER" button.
- A panel appears asking the user to enter the Book ID.
- The user enters a valid Book ID.
- The user clicks show button.
- The system checks whether the Book ID exists.
- If the book exists, the system retrieves the book's details and displays them.
- The cover image is loaded from the database or file system and shown on the screen.

Postcondiiton:

- The book's cover image is shown along with its details.

Exceptions:

- If the Book ID does not exist, the system displays the message: **"Book not found."**
- If the book has no associated cover image, the system shows a message: **"No cover image available."**
- If the Book ID field is empty, the system prompts the user to enter a valid ID.

■ Basic Use Case Form 1

Use case name: Display Book Info

Participating actors: Type-1 User(Admin), Type-2 User

Description: Allows users to view detailed information about a book by entering its Book ID.

Trigger: User clicks the 'Display Book Info' button.

Preconditions:

- User must be logged in.
- A valid Book ID must be entered.

■ Basic Use Case Form 2

Use case name: Update Book

Participating actors: Type-1 User(Admin), Type-2 User

Description: Allows the Type-1 User(Admin) to update book information after retrieving it by Book ID.

Trigger: Type-1 User(Admin) clicks the 'Update Book' button.

Preconditions:

- Type-1 User(Admin) must be logged in.
- The book to be updated must exist in the system.

■ Basic Use Case Form 3

Use case name: Display Favorite Books

Participating actors: Type-1 User(Admin), Type-2 User

Description: Displays a list of books that the user has marked as 'read' with a rating of 4 or 5.

Trigger: User clicks the 'Favorite Books' button.

Preconditions:

- User must be logged in.
- The book must be marked as "read" and must have a rating.

■ Basic Use Case Form 4

Use case name: Search Author by Name

Participating actors: Type-1 User(Admin), Type-2 User

Description: Allows users to search for an author by entering the name and surname.

Trigger: User clicks the 'Search Author' button.

Preconditions: User must be logged in. A valid name and surname must be provided.

■ Basic Use Case Form 5

Use case name: Display Favorite Authors

Participating actors: Type-1 User(Admin), Type-2 User

Description: Displays authors who have written books more than 3.

Trigger: User clicks the 'Favorite Authors' button.

Preconditions:

- User must be logged in.

-There must be more than 3 books in the system that written by the author.

■ Basic Use Case Form 6

Use case name: Display Unread Books

Participating actors: Type-1 User(Admin), Type-2 User

Description: Displays a list of books marked as 'not read' or 'wish to read' by the user.

Trigger: User clicks the 'Unread Books' button.

Preconditions:

- User must be logged in.
- The database must contain books with read status 2 or 3.

■ Basic Use Case Form 7

Use case name: Login

Participating actors: Type-2 User

Description: Allows users to log in with a valid username and password to access the system.

Trigger: User enters credentials and clicks the 'Login' button.

Preconditions:

- User must have a registered account in the system.

3.4.4 Relationships among Actors and Use Cases

The **Type-1 User(Admin)** has access to all functionalities, including adding, updating, and deleting books, as well as searching for authors and viewing book details.

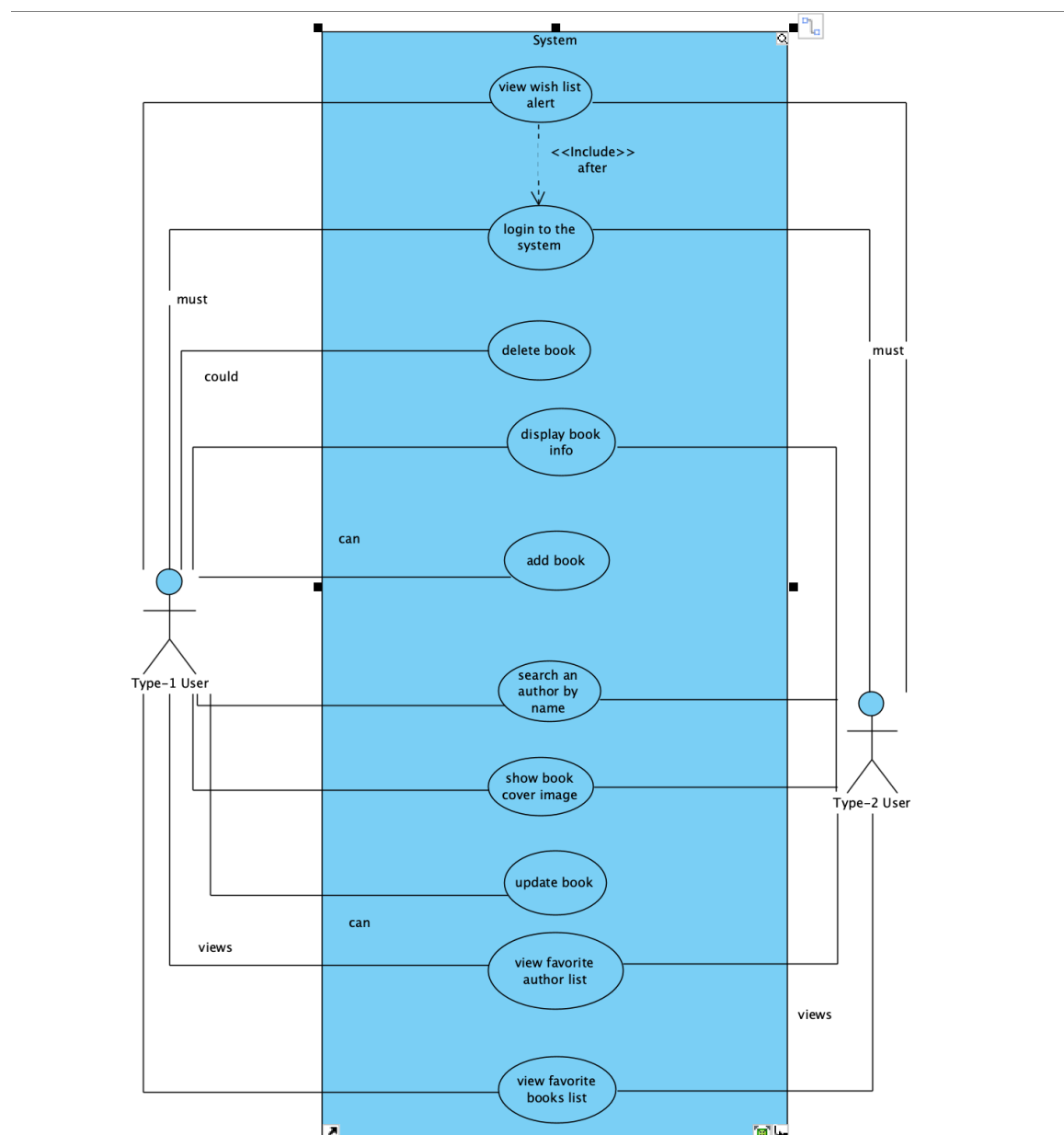
The **Type-2 User** has limited access and can only perform actions such as logging in, viewing favorite and unread books, and searching for authors.

Both Type-1 User(Admin) and Type-2 User can use features like searching and viewing book information, including “Display Book Info”, “Display Favorite Books”, and “Search Author by Name”.

The **System** is responsible for handling all user and Type-1 User(Admin) interactions, such as login validation, data retrieval, and presenting relevant outputs to the user interface.

3.4.5 Use Case Diagram

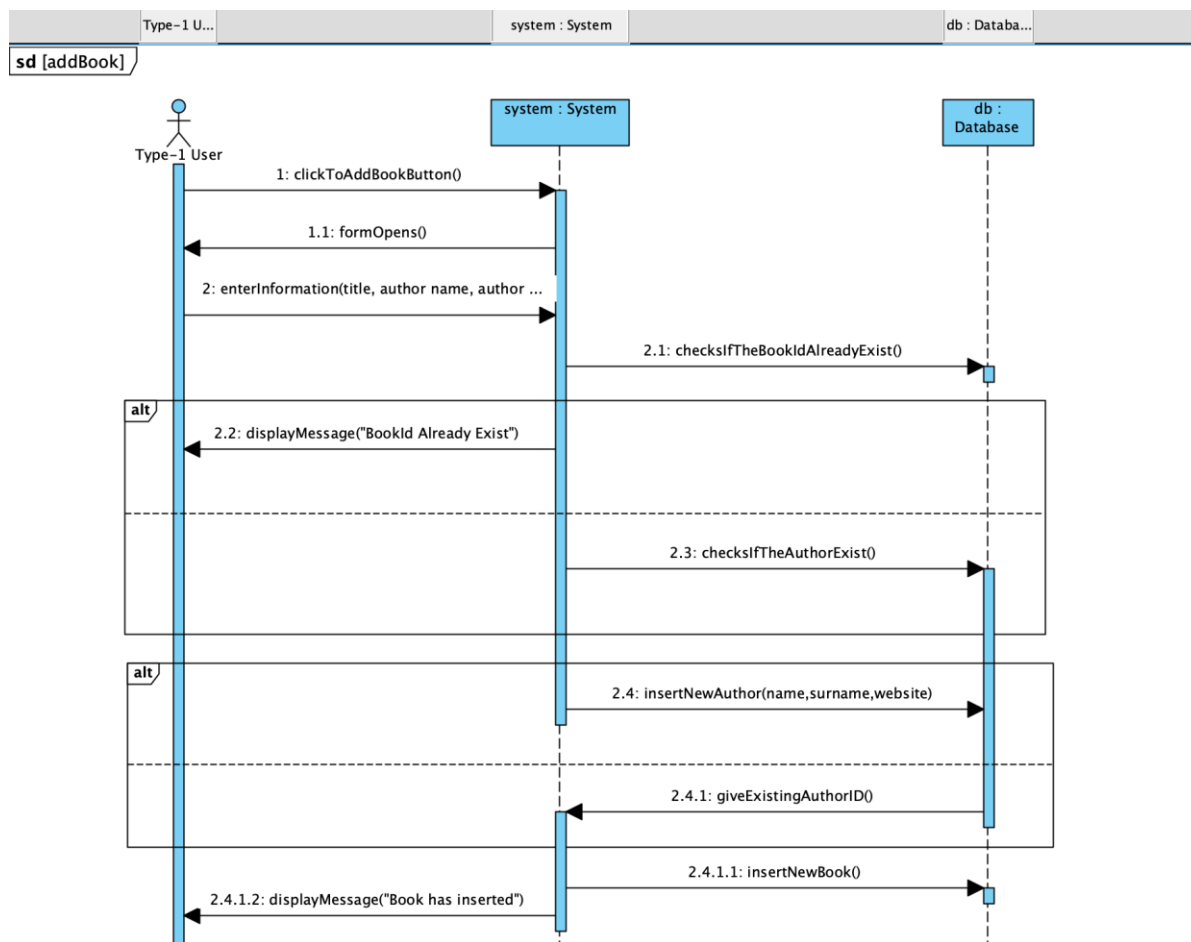
This use case diagram represents the functional interactions within the Library System, involving two primary user types: Type-1 User (Admin) and Type-2 User (Regular User). Both user types can log into the system, search authors by name, view book details, see cover images, and access their favorite books and authors lists. Upon login, users automatically receive wish list alerts, modeled using an <<include>> relationship. Type-1 Users have additional privileges such as adding new books, updating existing ones, and deleting records, reflecting their administrative role. The diagram clearly separates common and role-specific functionalities, providing a structured overview of user interactions with the system.



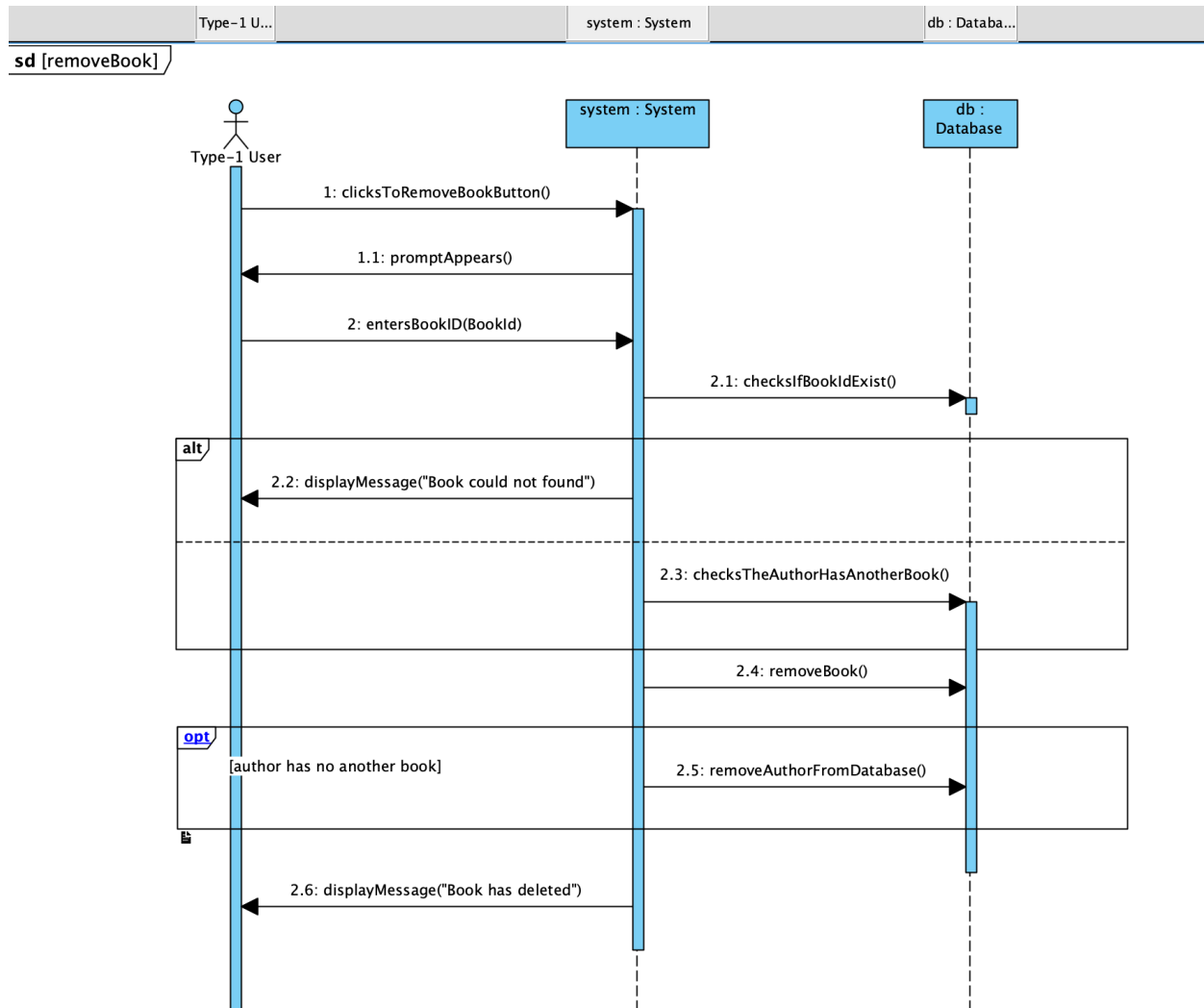
4 Behavioral Models

4.1 Sequence Diagram

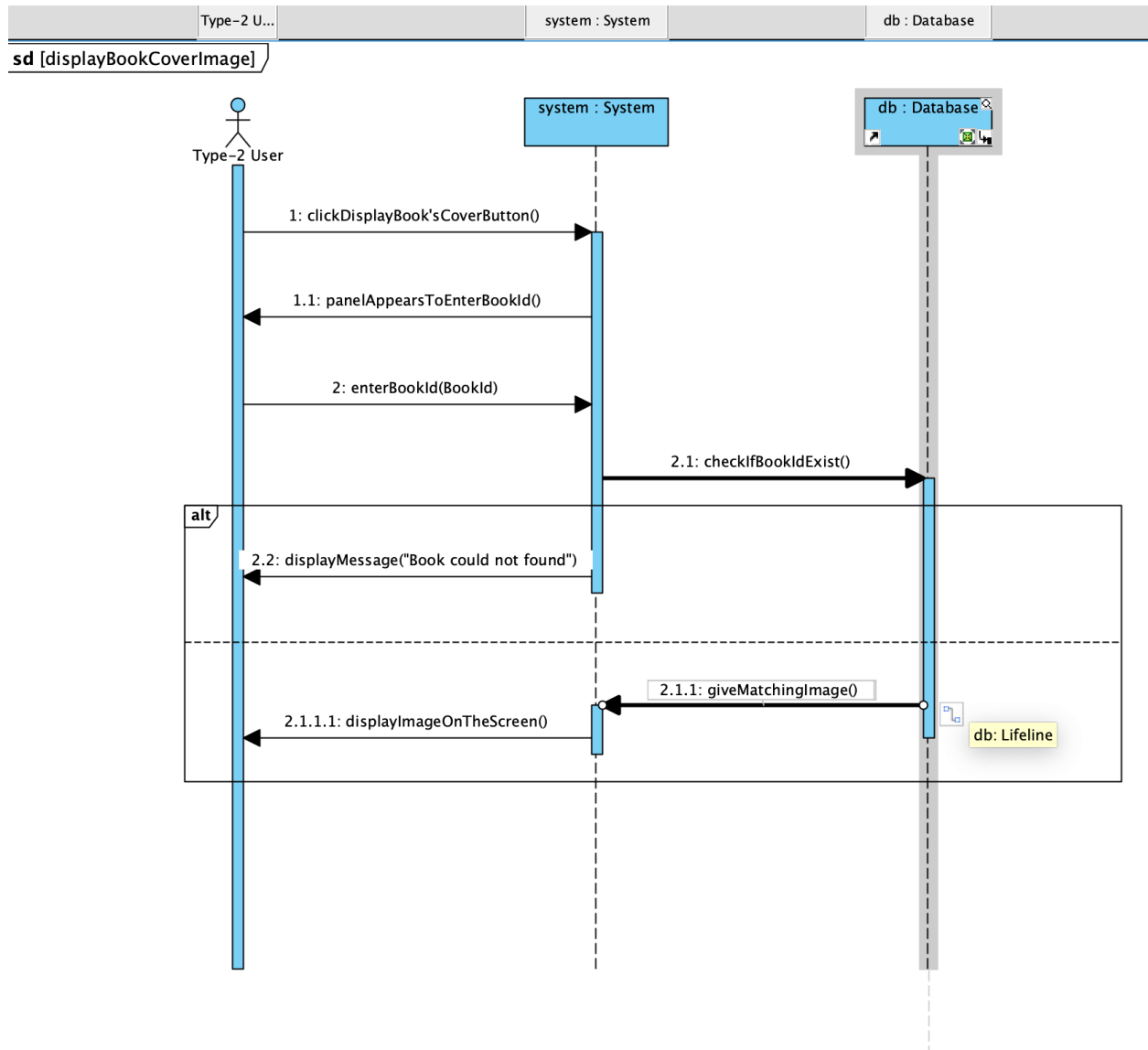
- 1) **Add Book Sequence Diagram:** This sequence diagram illustrates the *Add Book* process for a Type-1 User (Admin). After clicking the "Add Book" button and entering book details, the system checks if the Book ID exists. If it does, an error is shown. If not, it verifies the author. If the author doesn't exist, they are inserted and assigned an ID. Finally, the book is added to the database and a success message is displayed. *alt* fragments are used to show decision points for book and author existence.



- 2) **Delete Book Sequence Diagram:** This sequence diagram models the process of deleting a book by a Type-1 User (Admin). After entering the Book ID, the system checks if the book exists. If not, an error message is shown. If it exists, the system deletes the book and then checks if the related author has any other books. If not, the author is also deleted. Finally, a confirmation message is displayed to the user. The diagram uses alt and opt fragments to represent conditional flows based on book and author existence.



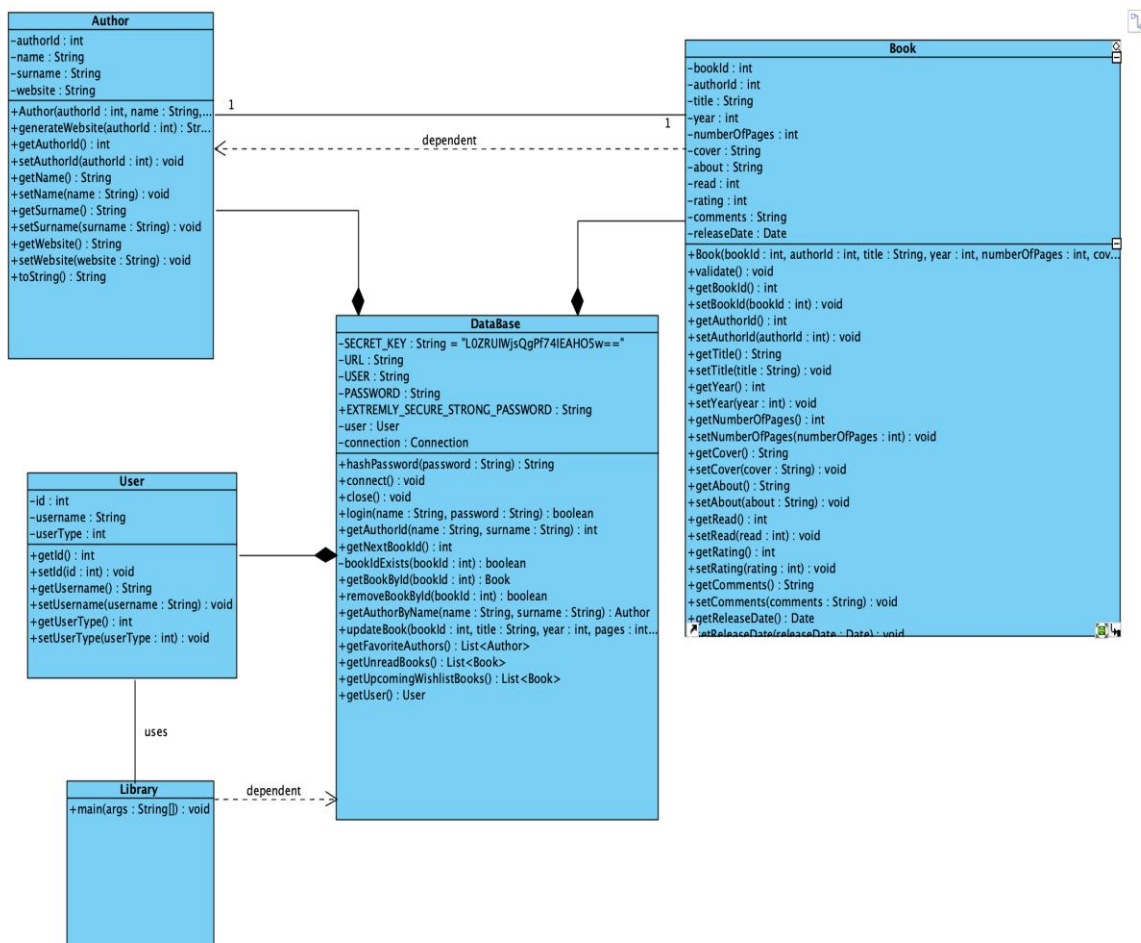
- 3) **Display Book Cover Image Sequence Diagram:** This sequence diagram models the process of displaying a book's cover image. After the user clicks the display button and enters a valid Book ID, the system checks if the book exists. If not, an error is shown. If the book is found, the system retrieves the cover image from the database and displays it. The diagram uses an alt fragment to represent the conditional flow based on the existence of the book.



5 Structural Models

5.1 Class Diagram

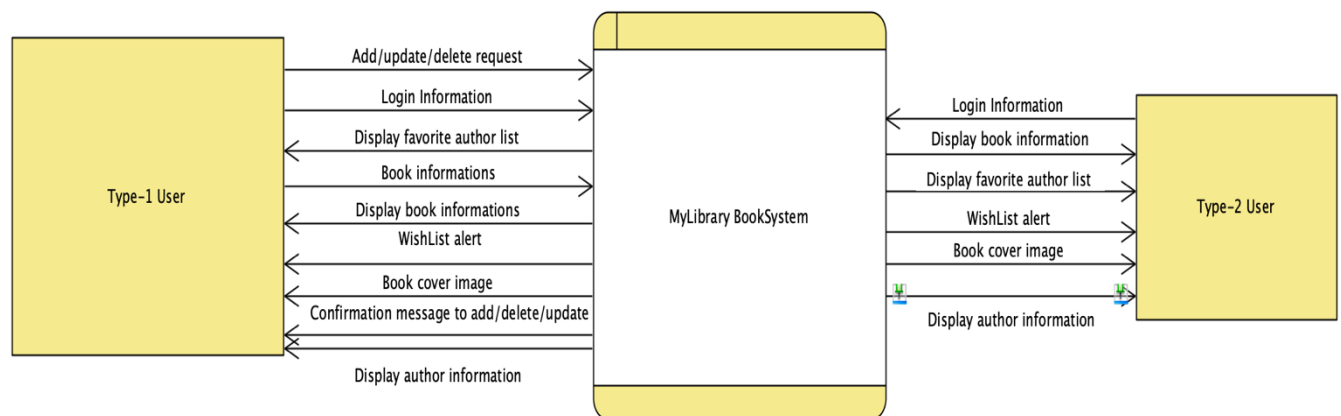
This UML class diagram represents the core structure of the MyLibrary Book System, designed to manage users, books, and authors within a Java-based application. At the center of the system is the Database class, which handles all major operations such as user login, book and author management, and database connectivity. The Book, Author, and User classes model the essential data entities, each encapsulating relevant attributes and behaviors. The Library class serves as the application's entry point and depends on the Database class to initiate functionality. Relationships are clearly defined using associations, aggregations, and dependencies, illustrating how each component interacts with the others to create a modular and maintainable system.



6 Process Modeling

6.1 Data Flow Diagram (DFD)

Context Diagram: This context diagram illustrates the external interaction between the MyLibrary BookSystem and its two main user types: Type-1 User (Admin) and Type-2 User. The system acts as a single high-level process, exchanging data flows with both user types. Type-1 Users can perform administrative tasks such as adding, updating, and deleting books, and receive confirmation messages in return. Both user types can log in, view book and author information, display cover images, see wish list alerts, and access favorite author lists. The data flows shown represent the inputs and outputs between users and the system without detailing internal processes, providing a high-level overview of system boundaries and interactions.

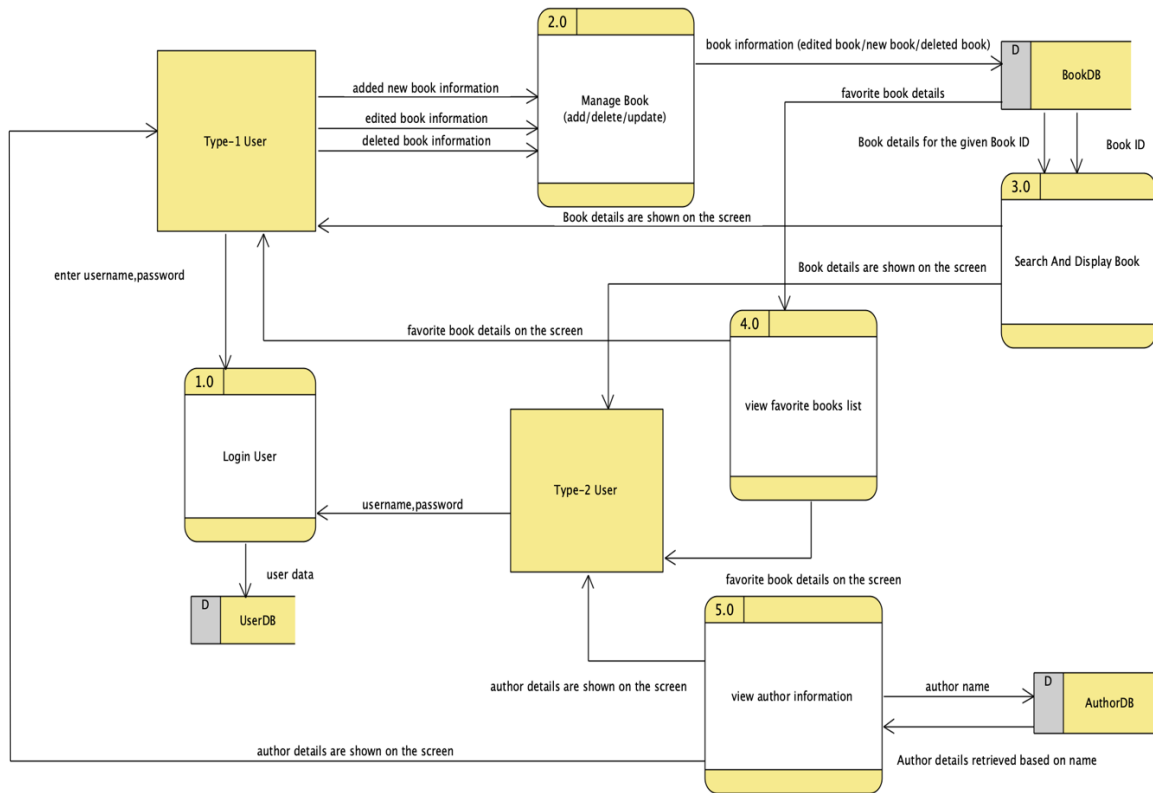


Level 0 DFD: This Level 0 Data Flow Diagram decomposes the MyLibrary Book System into its core processes and illustrates data interactions between external users, internal processes, and data stores.

- Both Type-1 User (Admin) and Type-2 User can log in through Process 1.0 – Login User, which authenticates credentials via the UserDB.

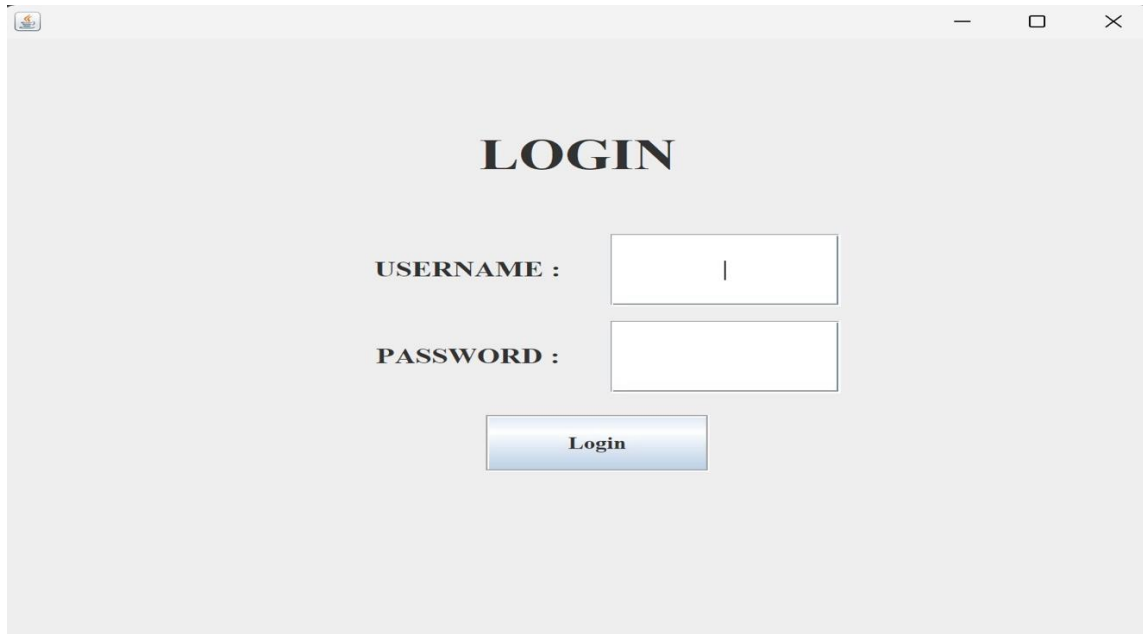
- Type-1 User has full privileges and can also perform the following:
 - Manage Books (2.0) — add, update, delete records stored in BookDB
 - Search and Display Book (3.0) — retrieve book details by Book ID
 - View Favorite Books List (4.0) — access favorite books from BookDB
 - View Author Information (5.0) — search for author details using name from AuthorDB.
- Type-2 User can access read-only operations such as viewing book/author details and wish list items via Processes 3.0, 4.0, and 5.0.
- Data flows like "Book details for the given Book ID", "Author details retrieved based on name", and "Favorite book details" illustrate the system's internal communication and output to the users.

This diagram shows a complete overview of user interactions and system responsibilities at a functional level.



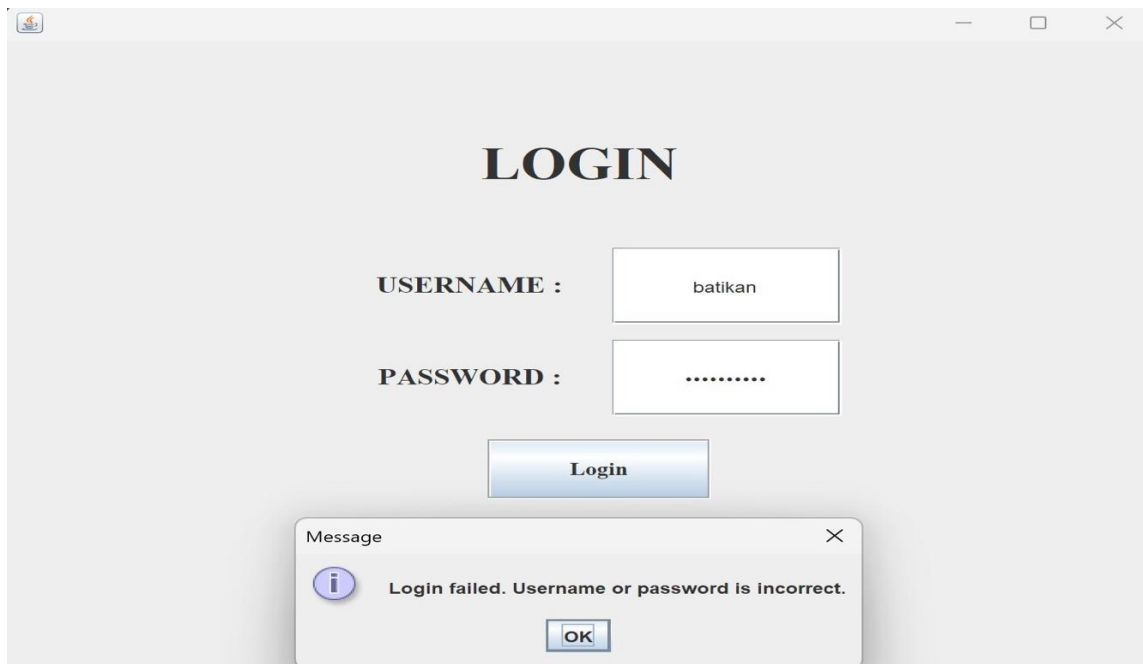
7 Graphical User Interface(s) (GUIs)

LOGIN PAGE:



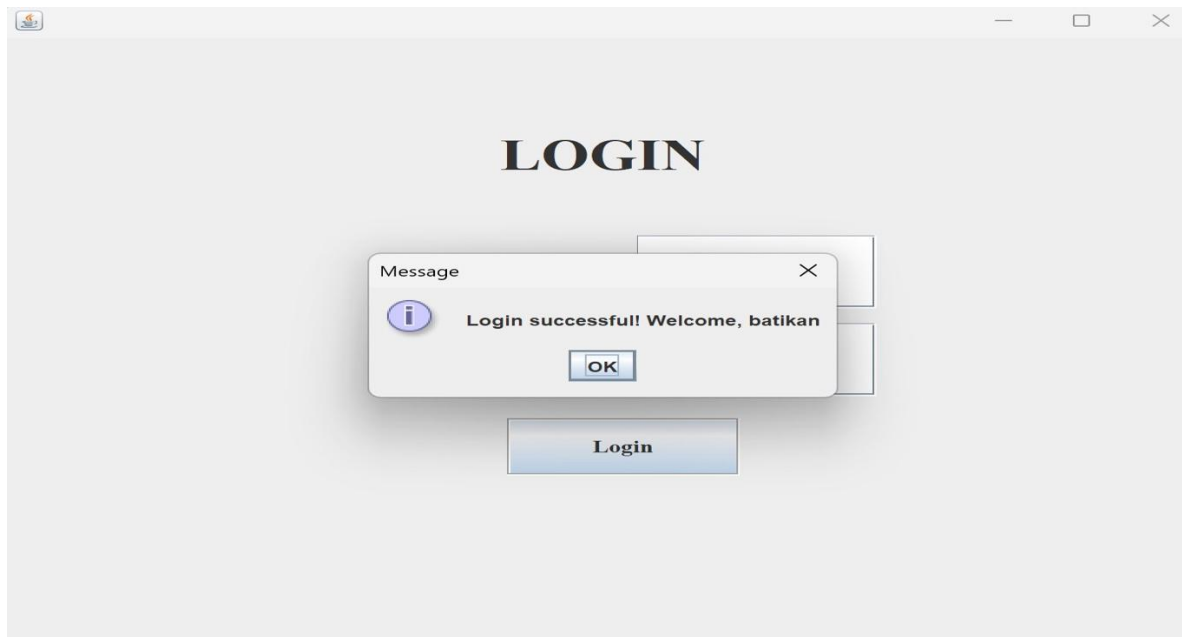
A screenshot of a graphical user interface window titled "LOGIN". The window has a light gray background and standard window controls (minimize, maximize, close) in the top right corner. In the center, the word "LOGIN" is displayed in a large, bold, black serif font. Below it, there are two input fields. The first is labeled "USERNAME :" and contains a single vertical bar cursor. The second is labeled "PASSWORD :" and is empty. Below the password field is a blue button with the text "Login" in white. The button has a slight gradient and a shadow.

INVALID LOGIN ATTEMPT:

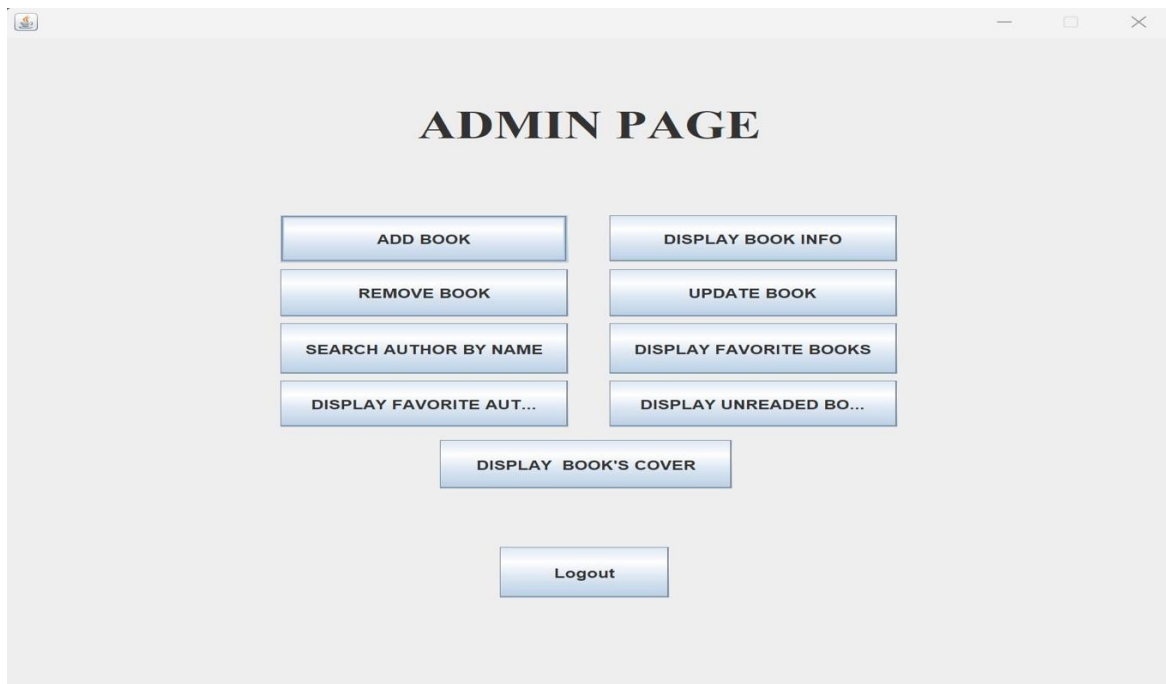


A screenshot of the same "LOGIN" window, but now showing an invalid login attempt. The "USERNAME :" field contains the text "batikan" and the "PASSWORD :" field contains a series of dots. The "Login" button is still present. Overlaid on the bottom of the window is a "Message" dialog box. The dialog box has a title bar with "Message" and a close button. It contains an information icon (a lowercase 'i' in a circle) and the text "Login failed. Username or password is incorrect." Below the text is an "OK" button.

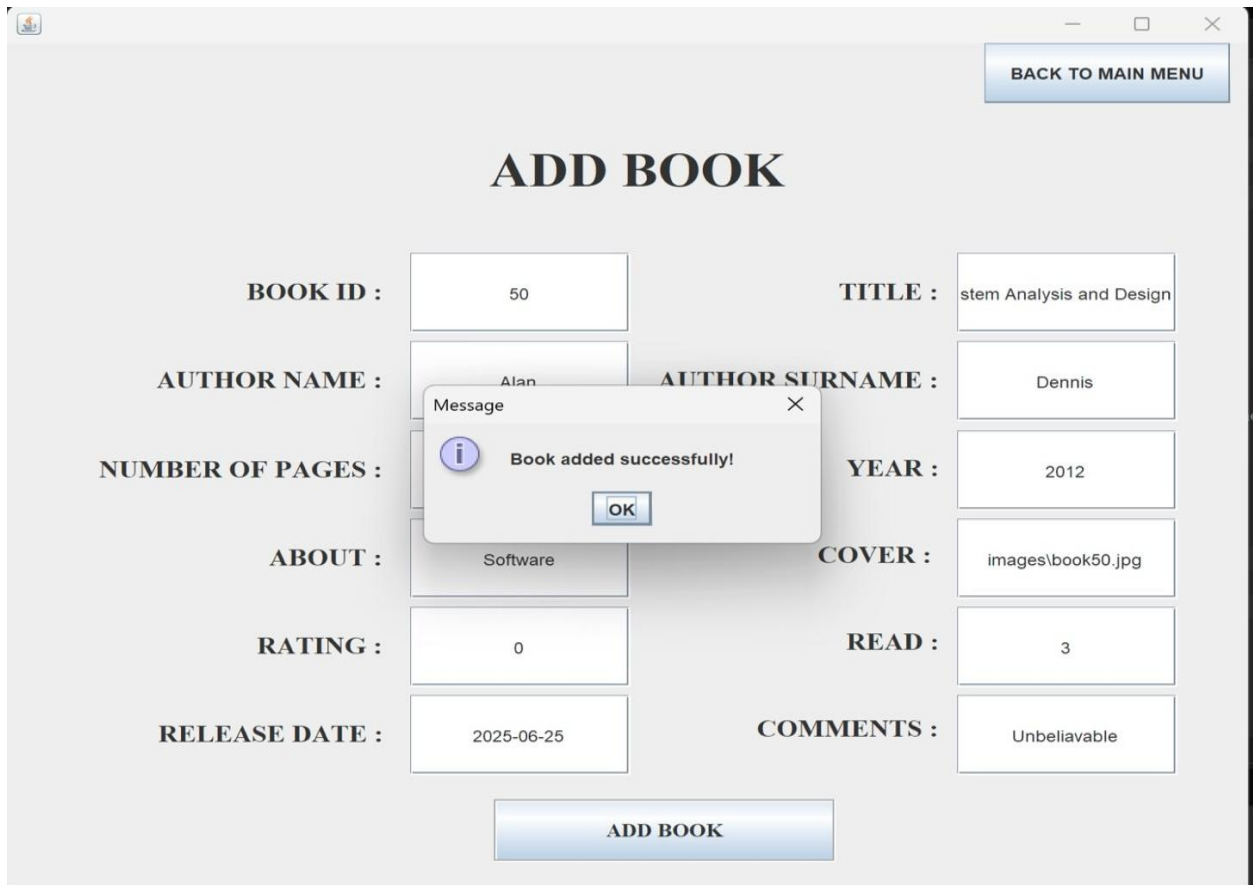
VALID LOGIN ATTEMPT:



ADMIN PAGE:



ADD BOOK PAGE:



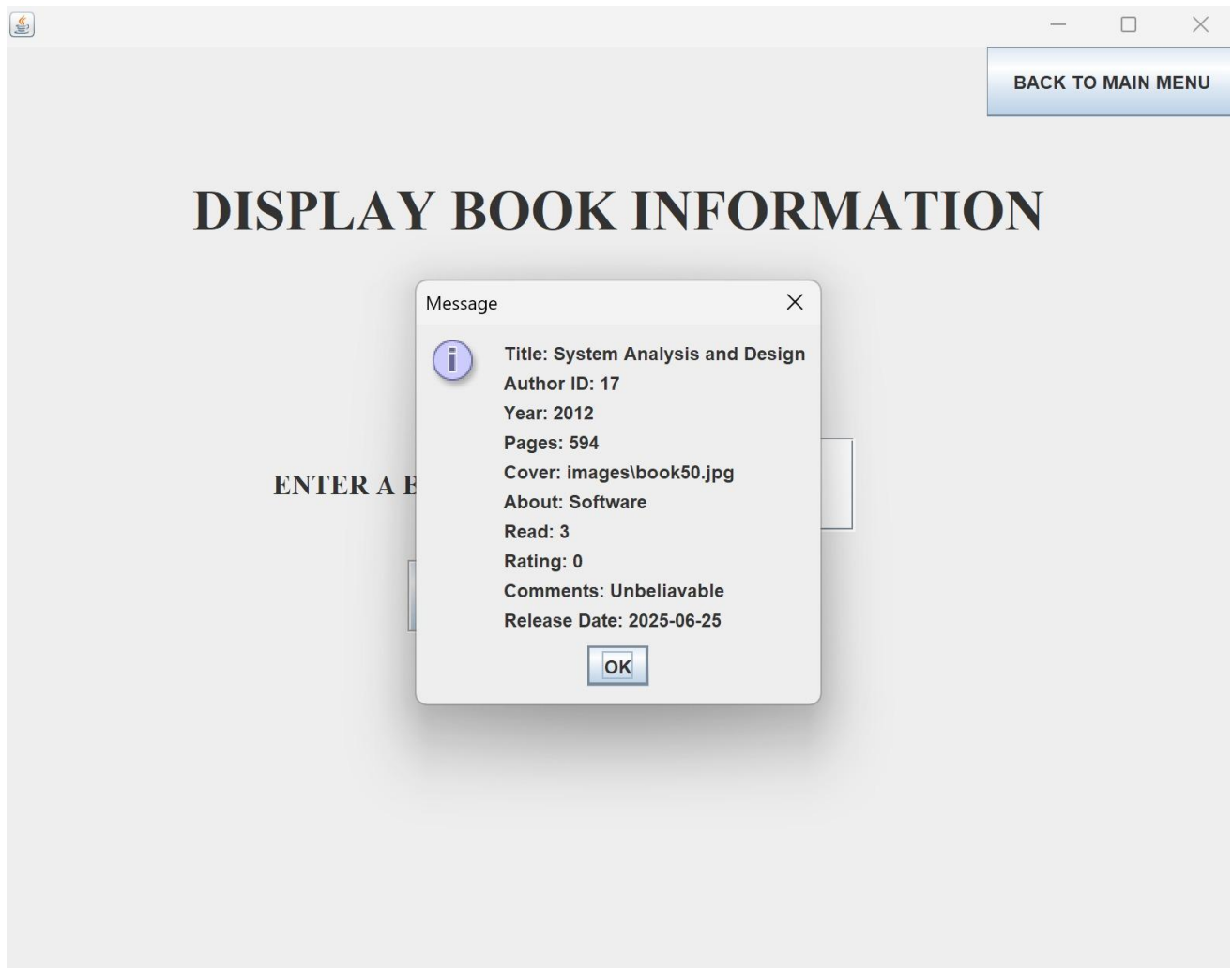
The screenshot shows a web application window titled "ADD BOOK". In the top right corner, there is a button labeled "BACK TO MAIN MENU". The main heading "ADD BOOK" is centered at the top. Below it, the form is organized into two columns. The left column contains labels and input fields for "BOOK ID" (50), "AUTHOR NAME" (Alan), "NUMBER OF PAGES" (empty), "ABOUT" (Software), "RATING" (0), and "RELEASE DATE" (2025-06-25). The right column contains labels and input fields for "TITLE" (stem Analysis and Design), "AUTHOR SURNAME" (Dennis), "YEAR" (2012), "COVER" (images\book50.jpg), "READ" (3), and "COMMENTS" (Unbellavable). At the bottom center is a large "ADD BOOK" button. A modal message box is overlaid in the center, displaying an information icon, the text "Book added successfully!", and an "OK" button.

BOOK ID :	50	TITLE :	stem Analysis and Design
AUTHOR NAME :	Alan	AUTHOR SURNAME :	Dennis
NUMBER OF PAGES :		YEAR :	2012
ABOUT :	Software	COVER :	images\book50.jpg
RATING :	0	READ :	3
RELEASE DATE :	2025-06-25	COMMENTS :	Unbellavable

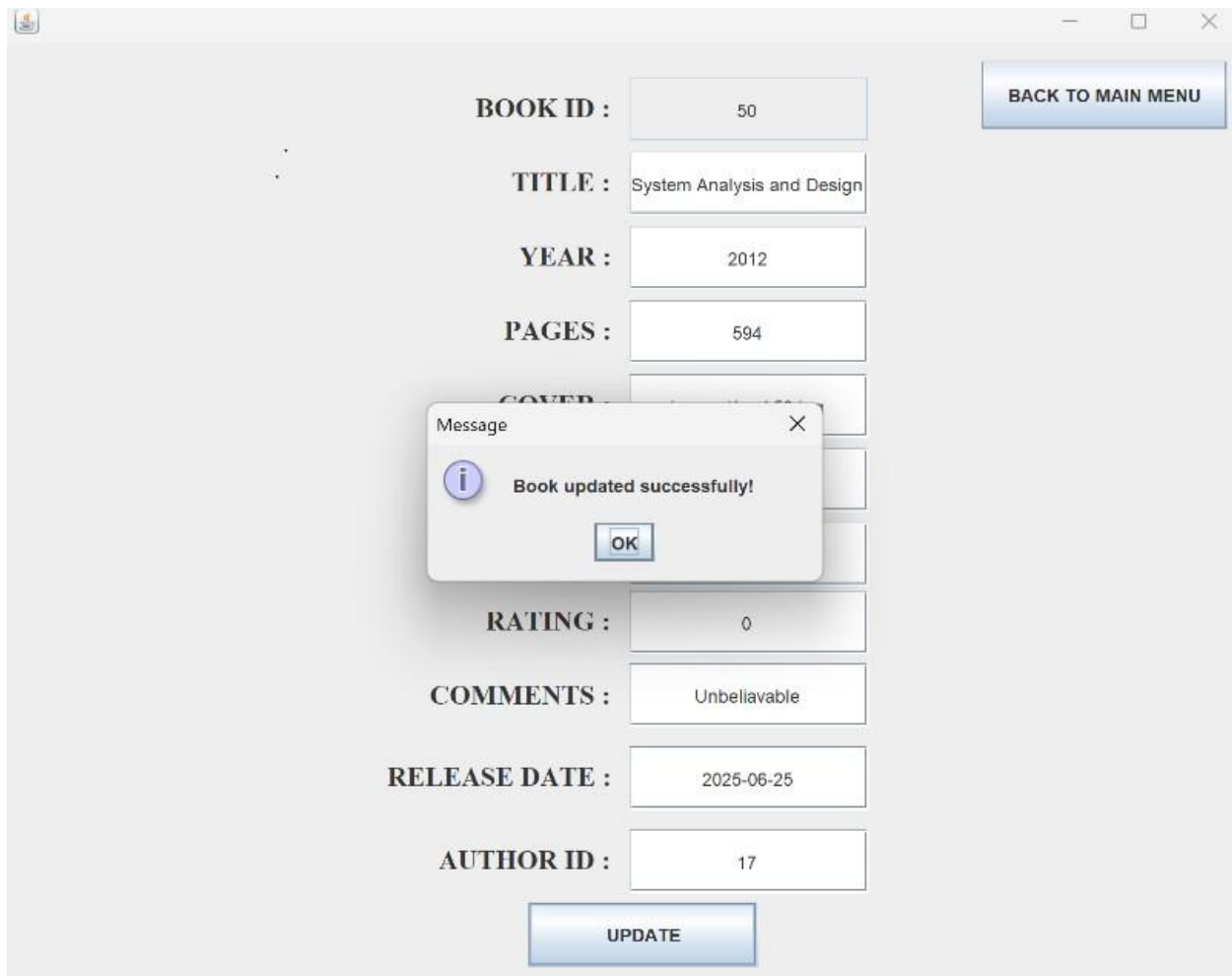
ADD BOOK

Message
Book added successfully!
OK

DISPLAY BOOK INFORMATION PAGE:



UPDATE BOOK PAGE:



The screenshot shows a web application window titled "UPDATE BOOK PAGE:". The form contains several input fields for book details. A modal message box is displayed in the center, indicating a successful update. The form fields are as follows:

Field Label	Value
BOOK ID :	50
TITLE :	System Analysis and Design
YEAR :	2012
PAGES :	594
COVER :	
RATING :	0
COMMENTS :	Unbelievable
RELEASE DATE :	2025-06-25
AUTHOR ID :	17

Buttons: "BACK TO MAIN MENU" (top right), "UPDATE" (bottom center).

Message Box:
Message
Book updated successfully!
OK

SEARCH AN AUTHOR BY NAME PAGE:

BACK TO MAIN MENU

SEARCH AN AUTHOR BY NAME

AUTHOR NAME : George

AUTHOR SURNAME : Orwell

SEARCH

SEARCHING AUTHOR BY NAME PAGE:

Book ID	Author ID	Title	Year	Pages	Cover	About	Read	Rating	Comments	Release D...
1	1	1984	1949	328	images/Bo...	Dystopian...	1	5	Brilliant!	
2	1	Animal Farm	1945	112	images/Bo...	Political s...	2	0		

FAVRİTE BOOKS LIST PAGE:

Favorite Books										
Book ID	Author ID	Title	Year	Pages	Cover	About	Read	Rating	Comments	Release ...
1	1	1984	1949	328	images/...	Dystopia...	1	5	Brilliant!	
4	3	The Hun...	2008	374	images/...	Dystopia...	1	5	Couldn't ...	
32	16	asd	123123	123	asd	asd	1	5		
96	13	asd	2011	124	asd	asd	1	5		
98	14	asd	2024	11	asd	1123	1	4		
123	15	asd	1412	123	asd	asd	1	5		

FAVORITE AUTHOR LIST PAGE:

Favorite Authors			
Author ID	Name	Surname	Website
13	Zeynep	Yıldız	website-13
15	asd	asd	website-15

UNREAD BOOKS LIST PAGE:

Unread Books										
Book ID	Author ID	Title	Year	Pages	Cover	About	Read	Rating	Comments	Release ...
2	1	Animal F...	1945	112	images/...	Political ...	2	0		
6	4	Franken...	1818	280	images/...	The mod...	2	0		
16	18	asdasd	123213	1212	asd	asd	2	0		
67	15	123	124	1254	asd	awds	2	0		

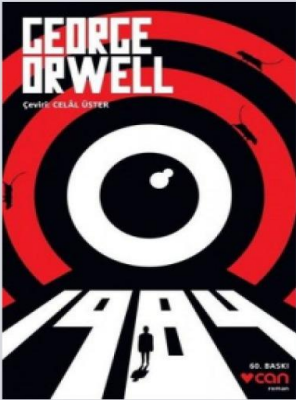
DISPLAY BOOK'S COVER PAGE:

ENTER A BOOK ID :

1

SHOW

BACK TO MAIN MENU



8 Conclusion and Future Work

The MyLibrary Book System project was developed as a desktop-based library management application using Java Swing for the graphical user interface and MySQL as the relational database backend. The system was designed to support two distinct user types: Type-1 Users (Admins), who have full access to book management operations (add, update, delete), and Type-2 Users (Regular Users), who can view books, search authors, and access their personalized favorite book and author lists. Throughout the project, we prioritized modular design, separation of concerns, and smooth user interaction through the GUI. All features were implemented based on clear use case scenarios, and the system was documented using UML diagrams such as use case, sequence, and data flow diagrams, allowing for a well-structured and maintainable software architecture.

From a security perspective, the login functionality was enhanced by hashing user passwords using SHA-256, a widely used and secure cryptographic hash function. This ensured that user credentials are not stored in plaintext in the database, providing a strong layer of protection against unauthorized access. Additionally, all input data was validated at both the GUI and database levels to reduce the risk of SQL injection and invalid data entries. By separating GUI logic, database access, and validation mechanisms, the project followed a well-designed, layered software architecture, making the system easier to extend, test, and maintain.

For future improvements, the system can be extended with features such as user registration with role assignment, book rating and review systems, reminder or alert functionalities for unread or upcoming books, and integration with external APIs (such as Google Books or OpenLibrary) to automatically fill in book metadata. The GUI can also be enhanced with search filters, sorting, and pagination for better usability. Moreover, transforming the application into a web-based or mobile-compatible platform would significantly improve accessibility and usability. Additional features such as session management, password reset functionality, and encryption of sensitive fields would also contribute to making the system more robust and production-ready.

Thanks to its modular design and adaptable architecture, this software product could be further customized and deployed as a functional library management system for a school or academic institution in the future.